

JavaScript Frameworks for Modern Web Development

**The Essential Frameworks, Libraries,
and Tools to Learn Right Now**

Second Edition

Sufyan bin Uzayr

Nicholas Cloud

Tim Ambler

Apress®

JavaScript Frameworks for Modern Web Development

Sufyan bin Uzayr
Al Manama, United Arab Emirates

Nicholas Cloud
Florissant, MO, USA

Tim Ambler
Nashville, TN, USA

ISBN-13 (pbk): 978-1-4842-4994-9
<https://doi.org/10.1007/978-1-4842-4995-6>

ISBN-13 (electronic): 978-1-4842-4995-6

Copyright © 2019 by Sufyan bin Uzayr, Nicholas Cloud, Tim Ambler

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Managing Director, Apress Media LLC: Welmoed Spahr
Acquisitions Editor: Louise Corrigan
Development Editor: James Markham
Coordinating Editor: Nancy Chen

Cover designed by eStudioCalamar

Distributed to the book trade worldwide by Springer Science+Business Media New York, 233 Spring Street, 6th Floor, New York, NY 10013. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail rights@apress.com, or visit <http://www.apress.com/rights-permissions>.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at <http://www.apress.com/bulk-sales>.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub via the book's product page, located at www.apress.com/9781484249949. For more detailed information, please visit <http://www.apress.com/source-code>.

Printed on acid-free paper

For Anza

—*Sufyan bin Uzayr*

Table of Contents

- About the Authors.....xv
- About the Technical Reviewerxvii
- Acknowledgmentsxix
- Introductionxxi
- Part I: Development Tools..... 1
- Chapter 1: Grunt 3
 - Installing Grunt..... 4
 - How Grunt Works 4
 - Gruntfile.js 5
 - Tasks 7
 - Plugins..... 8
 - Configuration 8
 - Adding Grunt to Your Project..... 8
 - Maintaining a Sane Grunt Structure 9
 - Working with Tasks 12
 - Managing Configuration 12
 - Task Descriptions 13
 - Asynchronous Tasks 14
 - Task Dependencies..... 15
 - Multi-Tasks 16
 - Multi-Task Options..... 18
 - Configuration Templates..... 19
 - Command-Line Options 20
 - Providing Feedback 21
 - Handling Errors..... 22

TABLE OF CONTENTS

Interacting with the File System	22
Source-Destination Mappings	23
Watching for File Changes.....	26
Creating Plugins.....	31
Getting Started	31
Creating the Task.....	32
Publishing to npm.....	36
Summary.....	36
Related Resources	37
Chapter 2: Yeoman.....	39
Installing Yeoman.....	40
Creating Your First Project	40
Subcommands.....	44
Creating Your First Generator	45
Yeoman Generators Are Node Modules	46
Sub-generators.....	47
Defining Secondary Commands	55
Composability	57
Summary.....	59
Related Resources.....	59
Chapter 3: PM2	61
Installation	61
Working with Processes.....	62
Recovering from Errors	65
Responding to File Changes	67
Monitoring Logs	68
Monitoring Resource Usage.....	70
Monitoring Local Resources	70
Monitoring Remote Resources	71

Advanced Process Management.....	75
JSON Application Declarations	75
Load Balancing Across Multiple Processors	81
Zero Downtime Deployments	84
Summary.....	87
Related Resources	87
Part II: Module Loaders	89
Chapter 4: RequireJS.....	91
Running the Examples	93
Working with RequireJS.....	94
Installation.....	94
Configuration	95
Application Modules and Dependencies.....	99
Paths and Aliases	104
Shims	108
Loader Plugins.....	114
Cache Busting	124
RequireJS Optimizer	126
Configuring r.js	126
Running the r.js Command	128
Summary.....	131
Chapter 5: Browserify.....	133
The AMD API vs. CommonJS.....	134
Installing Browserify	135
Creating Your First Bundle	136
Visualizing the Dependency Tree	138
Creating New Bundles As Changes Occur.....	139
Watching for File Changes with Grunt.....	139
Watching for File Changes with Watchify	140

TABLE OF CONTENTS

Using Multiple Bundles 142

The Node Way 147

 Module Resolution and the NODE_PATH Environment Variable..... 147

 Dependency Management..... 152

Defining Browser-Specific Modules..... 153

Extending Browserify with Transforms 155

 brfs 155

 folderify 156

 bulkify..... 157

 Browserify-Shim..... 158

Summary..... 160

Related Resources 160

Part III: Client-Side Frameworks 161

Chapter 6: Knockout..... 163

 Views, Models, and View Models 164

 The Recipe List 167

 Recipe Details..... 171

 Binding View Models to the DOM..... 175

 View Models and Forms..... 177

 Switching to “Edit” Mode 178

 Changing the Recipe Title 182

 Updating Recipe Servings and Cooking Time 183

 Adding and Removing Ingredients 187

 Instructions 193

 Citation..... 195

 Custom Components..... 196

 The Input List View Model 197

 The Input List Template 199

 Registering the Input List Tag..... 201

Subscribables: Cheap Messaging	204
Summary.....	208
Resources	208
Chapter 7: Angular.....	209
Differences Between Angular and AngularJS	209
Getting Started with Angular.....	211
Installation.....	211
Creating a Workspace in Angular	213
Directory Structure	215
Serving the App	216
Customizing the App.....	217
Dependency Injection in Angular.....	221
Creating and Registering an Injection Service	222
Conclusion	223
Part IV: Server-Side Frameworks	225
Chapter 8: Kraken.....	227
Environment-Aware Configuration.....	229
Shortstop Handlers.....	234
Configuration-Based Middleware Registration	239
Event Notifications.....	243
Structured Route Registration.....	244
Index Configuration	245
Directory Configuration	247
Routes Configuration	249
Dust Templates	251
Context and References	252
Sections.....	256
Iteration	256
Conditionality.....	258
Partials	259

TABLE OF CONTENTS

Blocks..... 261

Filters 262

Context Helpers 264

Dust Helpers 275

Let's Get Kraken 281

Summary..... 306

Related Resources 306

Part V: Managing Database Interaction 307

Chapter 9: Mongoose..... 309

Basic MongoDB Concepts 309

A Simple Mongoose Example..... 314

 Creating a Mongoose Schema for JSON Data 314

 Importing Data with Mongoose 317

 Querying Data with Mongoose 320

Working with Schemas 323

 Data Types 323

 Nested Schemas..... 325

 Default Property Values 326

 Required Properties..... 327

 Secondary Indexes 328

 Schema Validation 329

 Schema References 334

 Schema Middleware..... 340

Working with Models and Documents 343

 Document Instance Methods..... 347

 Document Virtuals 350

 Static Model Methods..... 352

Working with Queries..... 354

 Model.find()..... 355

 Model.findById()..... 357

 Model.findByIdAndUpdate()..... 360

Model.findByIdAndRemove()	361
Model.count()	362
Query.populate()	363
Finding Documents with Query Operators	365
Summary	375
Chapter 10: Knex and Bookshelf	377
Knex	378
Installing the Command-Line Utility	379
Adding Knex to Your Project	379
Configuring Knex	380
The SQL Query Builder	381
Migration Scripts	392
Seed Scripts	398
Bookshelf	400
What Is Object-Relational Mapping?	401
Creating Your First Bookshelf Model	402
Relationships	413
Summary	426
Related Resources	426
Part VI: Managing Control Flow	427
Chapter 11: Async.js	429
Sequential Flow	431
Parallel Flow	434
Pipeline Flow	437
Reusing a Pipeline	440
Loop Flow	443
Looping While Some Condition Remains True	443
Looping Until Some Condition Becomes False	446
Retry Loops	449
Infinite Loops	451

TABLE OF CONTENTS

Batch Flow 452

 Asynchronous Queue 453

 Asynchronous Cargo..... 455

Summary..... 458

Part VII: Further Useful Libraries 461

Chapter 12: Underscore and Lodash..... 463

 Installation and Usage..... 465

 Aggregation and Indexing 466

 countBy() 467

 groupBy() 468

 indexBy()..... 470

 Being Choosy 472

 Selecting Data from Collections 472

 Selecting Data from Objects..... 475

 Chaining 481

 Function Timing 485

 defer() 486

 debounce()..... 488

 throttle()..... 490

 Templates..... 493

 Loops and Other Arbitrary JavaScript in Templates..... 495

 Living Without Gator Tags 497

 Accessing the Data Object Within a Template 499

 Default Template Data 501

Summary..... 503

Related Resources 503

Part VIII: Front-End Development	505
Chapter 13: React	507
React Overview	507
What Makes React Special?	508
Getting Started with React.....	509
How to Add React to Web Pages?.....	509
Installation	510
Building a To-Do Application	514
Summary.....	520
Chapter 14: Vue.js	523
Vue.js Overview.....	523
What Is Vue.js?	523
What Is Vue Meant For?.....	524
Getting Started with Vue.js.....	526
Installation	526
Building Our First Vue App	528
Digging Deeper	532
Directory Structure	532
src/main.js File	533
src/App.vue File.....	534
components/HelloWorld.vue File	535
public/index.html File	538
Summary.....	539
Next Steps	539
Index.....	541

About the Authors



Sufyan bin Uzayr is a web developer with over 10 years of experience in the industry. He specializes in a wide variety of technologies, including JavaScript, WordPress, Drupal, PHP, and UNIX/Linux shell and server management, and is the author of five previous books. Sufyan is the Director of Parakozm, a multinational design and development consultancy firm that offers customized solutions to a global clientele. He is also the CTO at Samurai Servers, a server management and security company catering mainly to enterprise-scale audience. He takes a keen interest in technology, politics, literature, history, and sports, and in

his spare time he enjoys teaching coding and English to students. Read more about his works at www.sufyanism.com.



Nicholas Cloud is a software developer who lives in the very humid city of St. Louis. For over a decade, he has forged his skills into a successful career. He has developed web applications, web services, and desktop software on diverse platforms with JavaScript, C#, and PHP. A strong proponent of open source software, Nicholas contributes to userland projects and has written several of his own open source libraries. He speaks at a variety of user groups and conferences and writes books, technical articles, and blog posts in his spare time. He opines on Twitter at @nicholasccloud.

ABOUT THE AUTHORS



Tim Ambler is a software engineer from Nashville, Tennessee. His passion for programming follows in the footsteps of his father, who introduced him to computers at a young age with a Commodore 64. Tim is the author of several popular open source projects, one of which (whenLive) has been featured by GitHub's staff. An occasional conference speaker and frequent writer, Tim has been referenced multiple times in online publications such as *JavaScript Weekly* and *Node Weekly*. He currently lives in the 12 South area with his wife, Laura, and two cats. You can follow him on Twitter at @tkambler.

About the Technical Reviewer



Aleemullah Samiullah is a seasoned developer with over 10+ years of experience in front-end technologies. He is a senior engineer at Software AG and has previously worked at Publicis Sapient and Infosys, gaining considerable expertise, working with major brands such as Tesco, Target, and Holt Renfrew. He enjoys crafting digital experiences based on human-centered design, with a keen focus on usability and accessibility. In his career, Aleem has used various JavaScript libraries, including React, Angular, Backbone, ExtJS, jQuery, and so on. He is passionate about the latest technologies and actively participates in JS meetups and conferences. When he's not coding, he likes to travel, write blog posts, and spend time with family.

Aleem can be found on LinkedIn at www.linkedin.com/in/aleemullah/.

Acknowledgments

There are several people who deserve to be on this page because this book would not have come into existence without their support. That said, some names deserve a special mention, and I am genuinely grateful to

- My mom and dad, for everything they have done for me
- Faisal Fareed and Sadaf Fareed, my siblings, for helping with things back home
- Nancy Chen, Content Development Editor for this book, for keeping track of everything and for being very patient as I kept missing one deadline or the other
- The Apress team, especially Louise Corrigan, Jade Scard, and James Markham, for ensuring that the book's content, layout, formatting, and everything else remains perfect throughout
- The coauthors of this book's first edition and the tech reviewer, for going through the manuscript and providing his insight and feedback
- Typesetters, cover designers, printers, and everyone else, for their part in the development of this book
- All the folks associated with Parakozm, either directly or indirectly, for their help and support
- The JavaScript community at large, for all their hard work and efforts

—Sufyan bin Uzayr

Introduction

They tell me we're living in an information age, but none of it seems to be the information I need or brings me closer to what I want to know. In fact (I'm becoming more and more convinced) all this electronic wizardry only adds to our confusion, delivering inside scoops and verdicts about events that have hardly begun: a torrent of chatter moving at the speed of light, making it nearly impossible for any of the important things to be heard.

—Matthew Flaming, *The Kingdom of Ohio*

The notion that “technology moves quickly” is a well-worn aphorism, and with good reason: technology does move quickly. But at this moment, JavaScript in particular is moving very quickly indeed—much like that “torrent of chatter moving at the speed of light” that Matthew Flaming refers to in *The Kingdom of Ohio*. The language is in the midst of what many have called a renaissance, brought about by the rapidly increasing sophistication of browser-based applications and the rising popularity of JavaScript on the server, thanks to Node.js.

An almost feverish pace of innovation is occurring within the JavaScript community that, while endlessly fascinating to follow, also presents some unique challenges of its own. JavaScript’s ecosystem of libraries, frameworks, and utilities has grown dramatically. Where once a small number of solutions for any given problem existed, many can now be found, and the options continue to grow by the day. As a result, developers find themselves faced with the increasingly difficult task of choosing the appropriate tools from among many seemingly good options.

If you’ve ever found yourself wondering why JavaScript seems to be attracting so much attention, as we have, it’s worth stopping for a moment to consider the fact that JavaScript, a language that was created by one person in 10 days, now serves as the foundation upon which much of the Web as we know it sits. A language that was originally created to solve relatively simple problems is now being applied in new and innovative ways that were not originally foreseen. What’s more, JavaScript is a beautifully expressive language, but it’s not without its share of rough edges and potential pitfalls.

INTRODUCTION

While flexible, efficient, and ubiquitous, JavaScript concepts such as the event loop and prototypal inheritance can prove particularly challenging for those coming to the language for the first time.

For these and many other reasons, the development community at large is still coming to terms with how best to apply the unique features that JavaScript brings to the table. We've no doubt only scratched the surface of what the language and the community behind it are capable of. For those with an insatiable appetite for knowledge and a desire to create, now is the perfect time to be a JavaScript developer.

We have written *JavaScript Frameworks for Modern Web Development* to serve as your guide to a wide range of popular JavaScript tools that solve difficult problems at both ends of the development stack: in the browser and on the server. The tutorials and downloadable code examples contained within this book illustrate the usage of tools that manage dependencies, structure code in a modular fashion, automate repetitive build tasks, create specialized servers, structure client-side applications, facilitate horizontal scaling, perform event logging, and interact with disparate data stores.

The libraries and frameworks covered include Grunt, Yeoman, PM2, RequireJS, Browserify, Knockout, Angular, Kraken, Mongoose, Knex, Bookshelf, Async.js, Underscore, Lodash, React, and Vue.js.

In writing *JavaScript Frameworks for Modern Web Development*, our goal was to create a filter for the “torrent of chatter” that often seems to surround JavaScript and, in so doing, to allow what we believe are some important things to be heard. We hope the information contained within these pages proves as useful to you as it has to us.

Who This Book Is For

This book is intended for web developers who are already confident with JavaScript, but also frustrated with the sheer number of solutions that exist for seemingly every problem. This book helps lift the fog, providing the reader with an in-depth guide to specific libraries and frameworks that well-known organizations are using right now with great success. Topics pertaining to both client-side and server-side development are covered. As a result, readers will gain the most benefit from this book if they already have at least an intermediate familiarity with both the web browser Document Object Model (DOM), common client-side libraries like jQuery, and Node.js.

How This Book Is Structured

This book covers a wide selection of JavaScript tools that are applicable throughout the entire development process, from a project's first commit to its first release and beyond. To that end, the chapters have been grouped into the following parts.

Part 1: Development Tools

Grunt

Larry Wall, the creator of Perl, describes the three virtues of a great programmer as laziness, impatience, and hubris. In this chapter, we'll focus on a tool that will help you strengthen the virtue of laziness—Grunt. This popular task runner provides developers with a framework for creating command-line utilities that automate repetitive build tasks such as running tests, concatenating files, compiling SASS/LESS stylesheets, checking for JavaScript errors, and more. After reading this chapter, you'll know how to use several popular Grunt plugins as well as how to go about creating and sharing your own plugins with the community.

Yeoman

Yeoman provides JavaScript developers with a mechanism for creating reusable templates ("generators") that describe the overall structure of a project (initially required dependencies, Grunt tasks, etc.) in a way that can be easily reused over and over. Broad community support also allows you to take advantage of a wide variety of preexisting templates. In this chapter, we'll walk through the process of installing Yeoman and using several popular preexisting generators. Finally, we'll take a look at how we can create and share our own templates with the community.

PM2

In this chapter, we will close out our discussion of development tools by taking a look at PM2, a command-line utility that simplifies many of the tasks associated with running Node applications, monitoring their status, and efficiently scaling them to meet increasing demand.

Part 2: Module Loaders

RequireJS and Browserify

JavaScript lacks a native method for loading external dependencies in the browser—a frustrating oversight for developers. Fortunately, the community has stepped in to fill this gap with two very different and competing standards: the Asynchronous Module Definition (AMD) API and CommonJS. We'll dive into the details of both and take a look at widely used implementations of each: RequireJS and Browserify. Each has its merits, which we'll discuss in detail, but both can have a profoundly positive impact on the way in which you go about structuring your applications.

Part 3: Client-Side Frameworks

Knockout and Angular

In recent years, web developers have witnessed a sharp rise in popularity of so-called “single-page apps.” Such applications exhibit behavior once available only on the desktop, but at the expense of increased code complexity within the browser. In this section, we'll dive into two widely used front-end frameworks that help minimize that complexity by providing proven patterns for solving frequently encountered problems: Knockout and Angular. Knockout focuses on the relationship between view and data, but otherwise leaves the application architecture and plumbing to the developer's discretion. Angular takes a more prescriptive approach, covering the view, data transfer, Dependency Injection, and so on.

Part 4: Server-Side Frameworks

Kraken

Client-side applications aren't very useful without a server with which to interact. In this chapter, we'll take a look at one popular framework that supports developers in the creation of back-end applications: Kraken.

Part 5: Managing Database Interaction

Mongoose, Knex, and Bookshelf

At the core of every application lies the most important component of any development stack—the data that our users seek. In this section, we’ll become familiar with two libraries that help simplify some of the complexity that’s often experienced when interacting with popular storage platforms such as MongoDB, MySQL, PostgreSQL, and SQLite. After reading this section, you’ll be comfortable defining schemas, associations, lifecycle “hooks,” and more.

Part 6: Managing Control Flow

Async.js

The asynchronous nature of JavaScript provides developers with a significant degree of flexibility—as opposed to forcing developers to execute their code in a linear fashion, JavaScript allows developers to orchestrate multiple actions simultaneously. Unfortunately, along with this flexibility comes a significant degree of additional complexity—what many developers refer to as “callback hell” or the “pyramid of doom.”

Part 7: Further Useful Libraries

A number of wonderfully useful libraries exist that this book would be remiss not to cover, but for which additional parts are not necessarily warranted. This part will cover such libraries.

Underscore and Lodash

Underscore (and its successor, Lodash) is an incredibly useful collection of functions that simplifies many frequently used patterns that can be tedious to implement otherwise. This brief chapter will bring these libraries to your attention, along with some of the more popular extensions that can also be included to enhance their usefulness even further. Examples are included that highlight some of the most frequently used portions of these libraries.

Part 8: Front-End Development

React and Vue.js

In this section, we will cover JavaScript frameworks that are geared for front-end development, such as React and Vue.js.

React, having the backing of Facebook, has risen in popularity in a very short span of time and continues to be a preferred choice for many developers.

On the other hand, Vue.js is a slightly less popular name in the field, but it has been gaining a steady and very loyal following, primarily due to its ease of use and simplicity.

Downloading the Code

Each chapter in this book contains many examples, the source code for which may be downloaded from www.apress.com/9781484249949 in zipped form.

Most examples are run with the Node.js runtime, which may be obtained from <https://nodejs.org>. Chapters with additional prerequisites will explain the necessary procedures for downloading and installing the examples. (For example, MongoDB is necessary to run examples in Chapter 9, which covers Mongoose.)

Any additional steps necessary for running code examples (e.g., executing curl requests) or interacting with a running example (e.g., opening a web browser and navigating to a specific URL) are explained alongside each listing.